

Receiving Kernel-Level Insights via eBPF: Can ABR Algorithms Adapt Smarter?

M. Ghasemi, D. Lorenzi, M. Dolati, F. Tashtarian, S. Gorinsky, C. Timmerer

Presenter: Farzad Tashtarian



WueWoWAS 2025

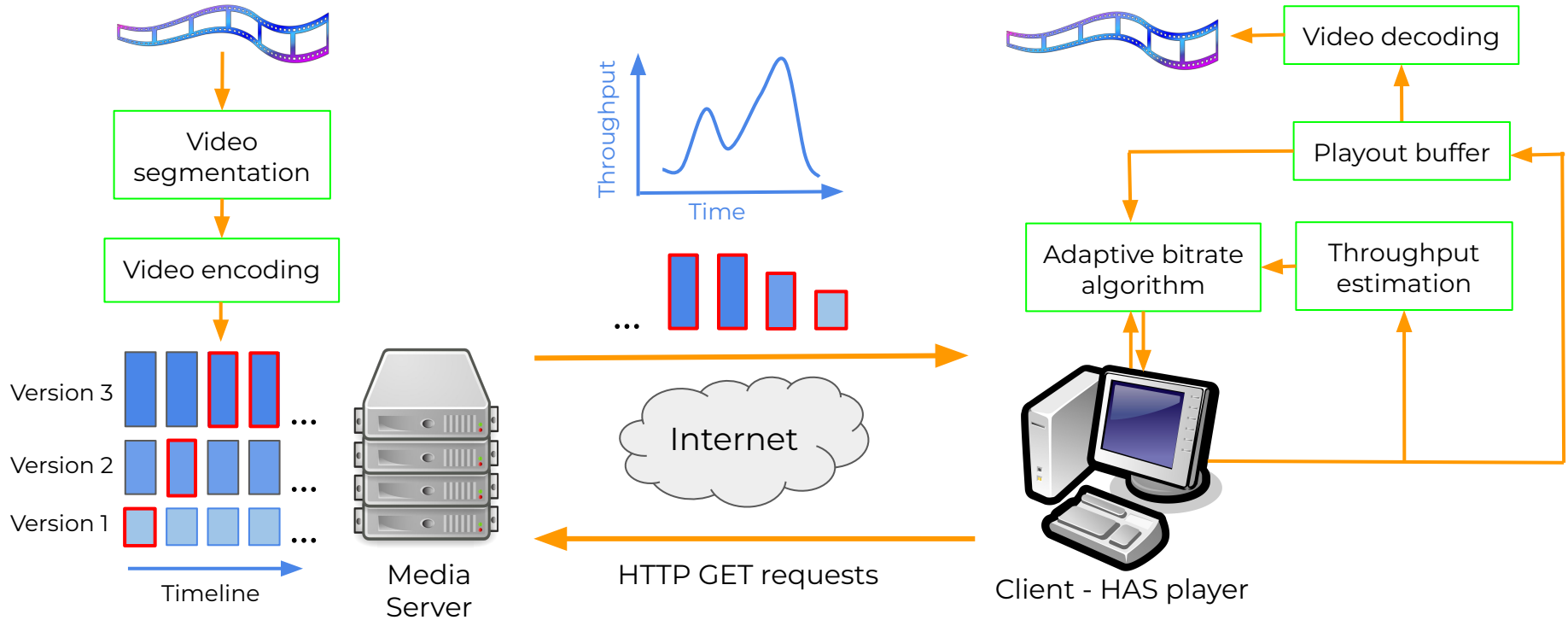
Oct. 7, 2025

Agenda

- Introduction
- Motivation and Initial Findings
- Proposed Method
- Evaluation and Results
- Conclusion
- Q&A

Introduction

HTTP Adaptive Streaming (HAS)

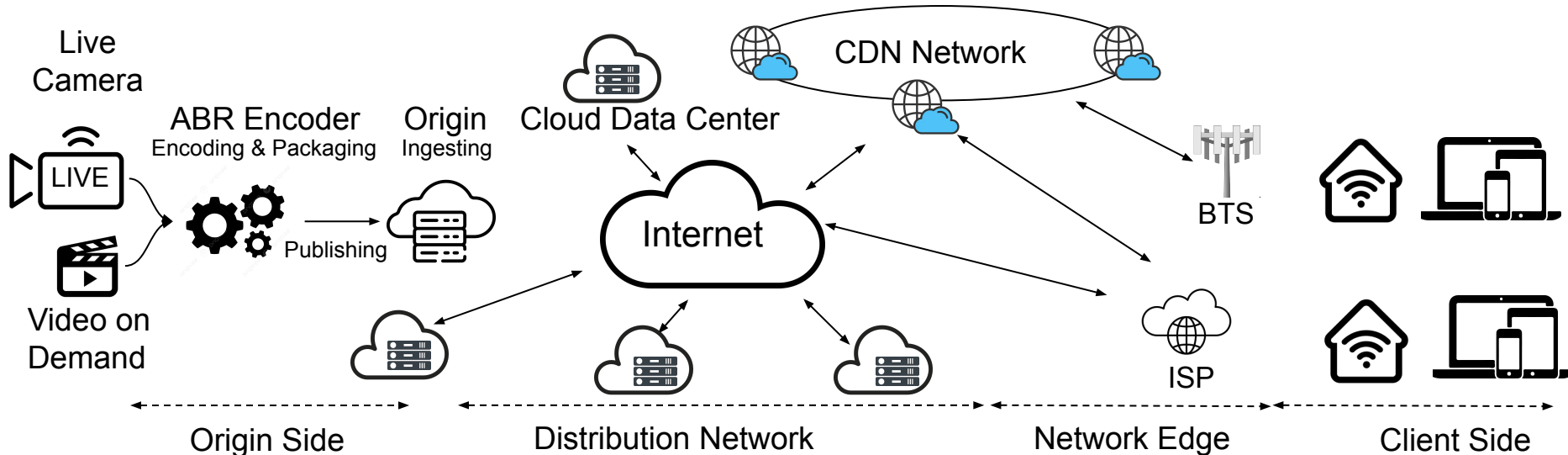


Video Streaming Pipeline

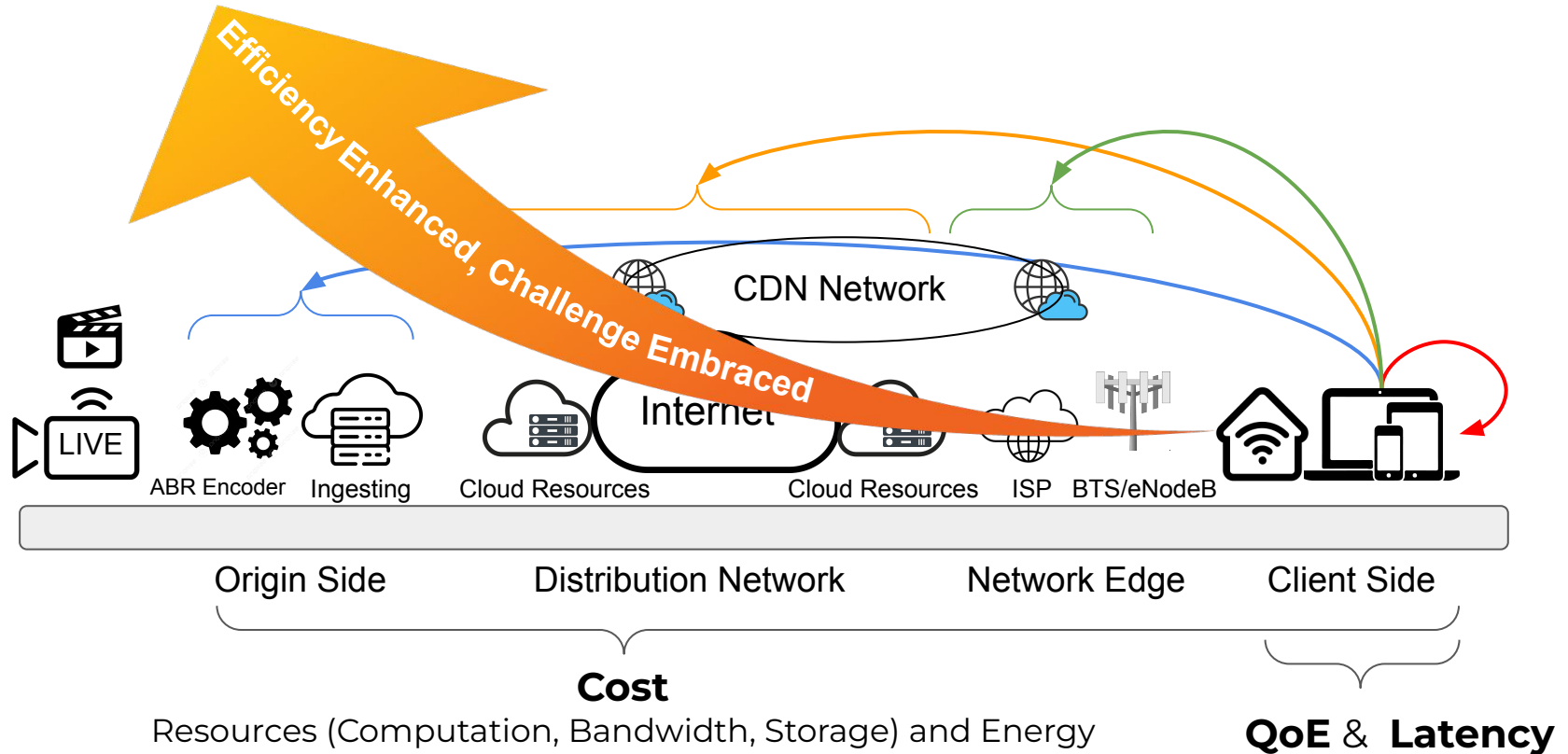
Content Provisioning

Content Delivery

Content Consumption

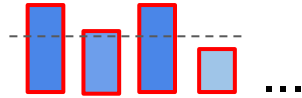


Degree of Freedom in Video Streaming



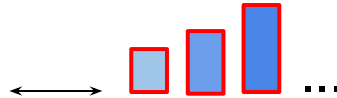
Quality of Experience (QoE)

Average quality (↑)



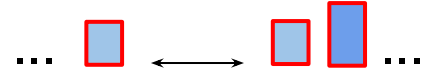
+

Start-up delay (↓)

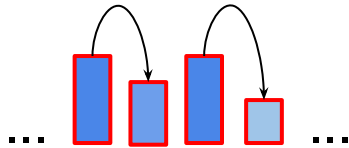


+

Stalls & stall duration (↓)

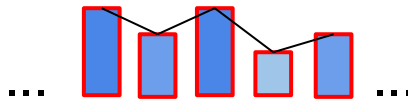


Quality switches (↓)



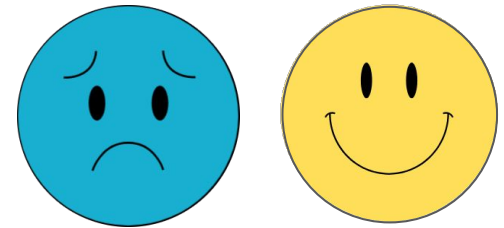
+

Video instability (↓)



=

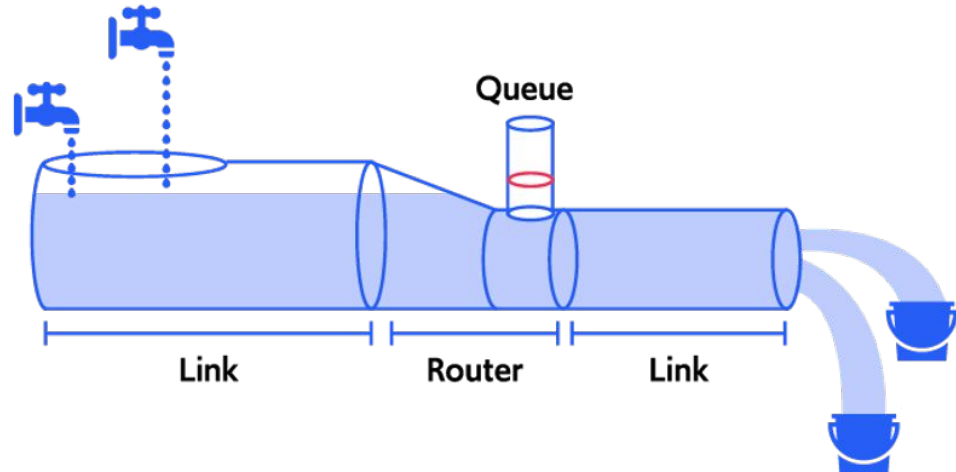
QoE (↑)



Motivation and Initial Findings

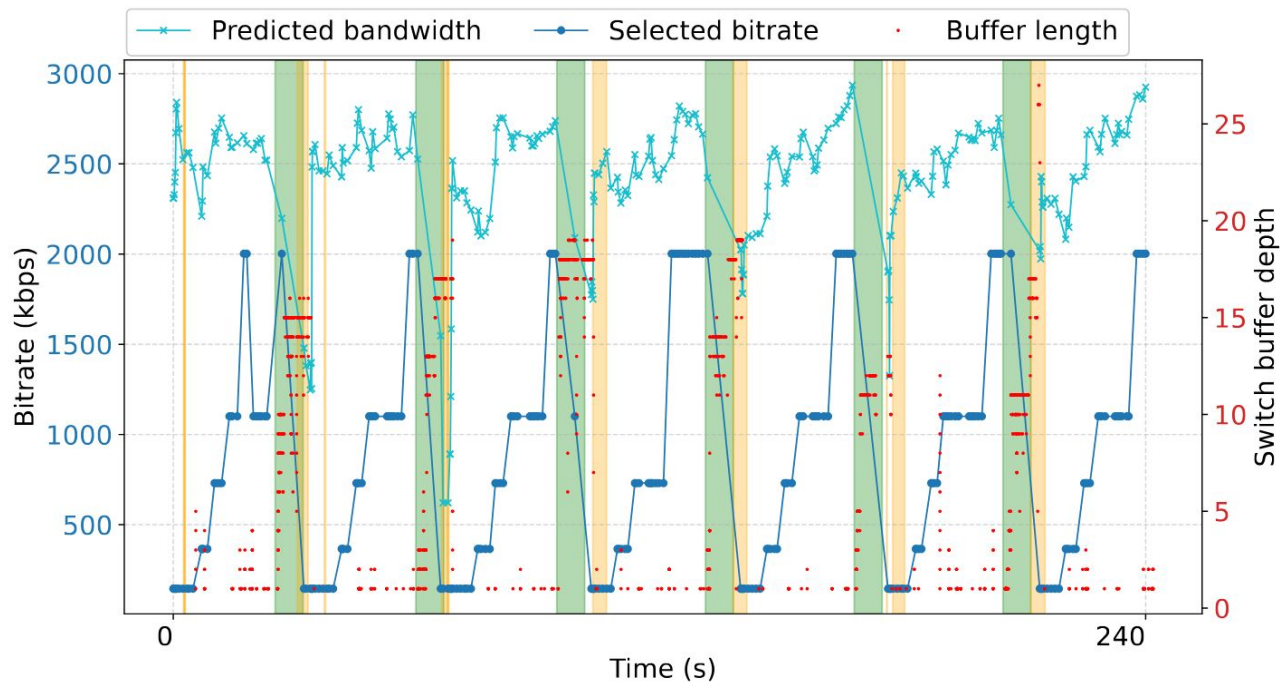
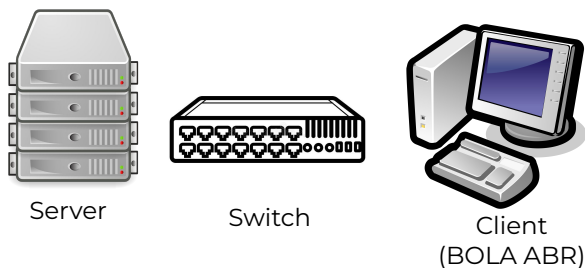
Network Competition Challenges

- Multiple clients competing for resources;
- Routers have limited space to handle incoming packets (queue);
- What happens if routers are overwhelmed with data?



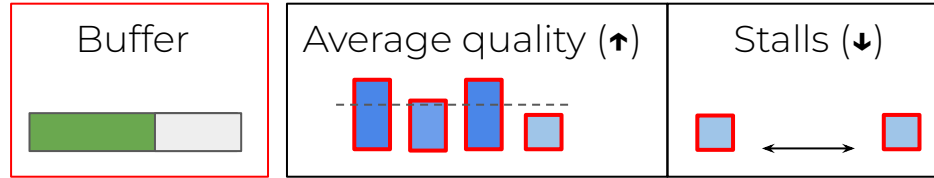
Initial Results

- Overloaded Switch for 30 s intervals during a 4 min session (simulated transient congestion)
- Green regions show induced **congestion**; orange highlights playback **stalls**; red dots represent the **queue lengths** of Switch.

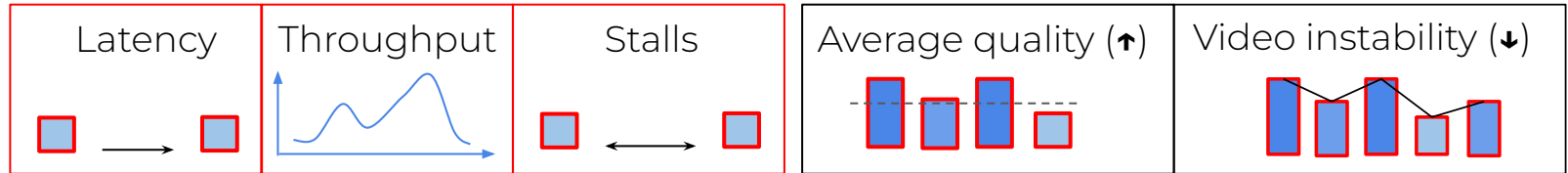


Initial Findings: ABR Algorithms

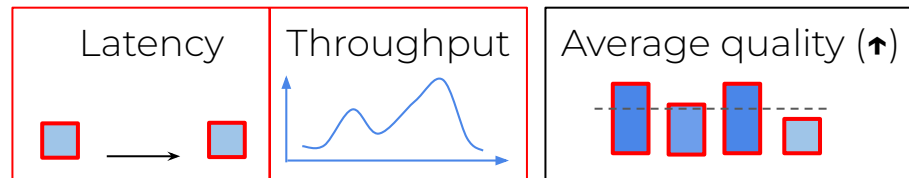
Buffer Occupancy based Lyapunov Algorithm (**BOLA**)



Low on Latency (**LoL+**)

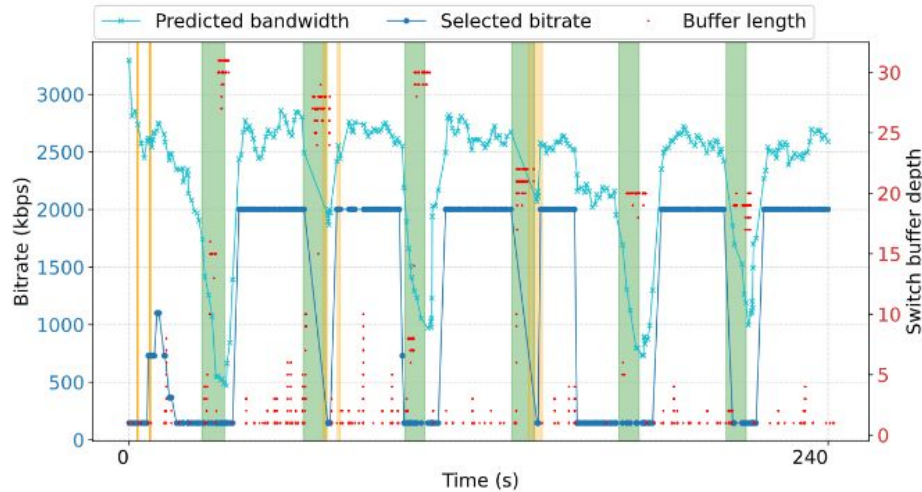


ThroughputRule

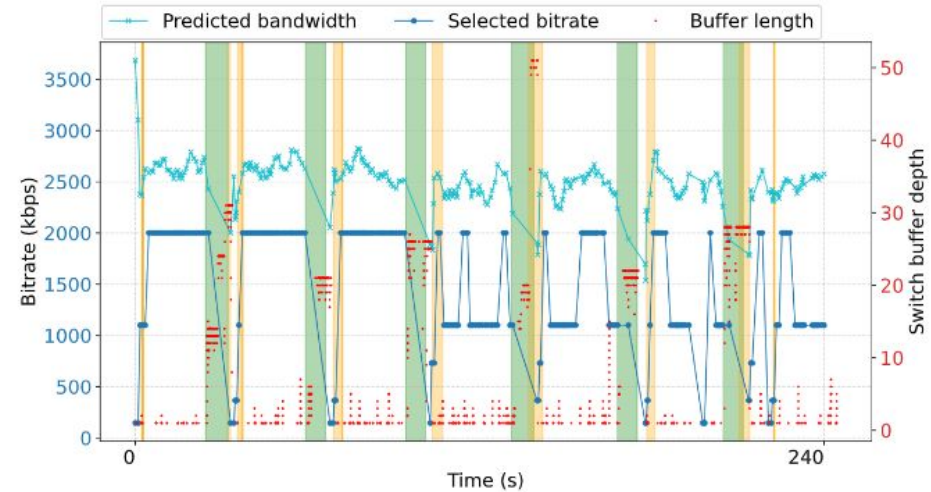


Initial Findings: Results

- Overloaded Switch for 30 s intervals during a 4 min session (simulated transient congestion)
- Green regions show induced congestion; orange highlights playback stalls; red dots represent the queue lengths of Switch.



LoL+



Throughput-based ABR

Proposed Method

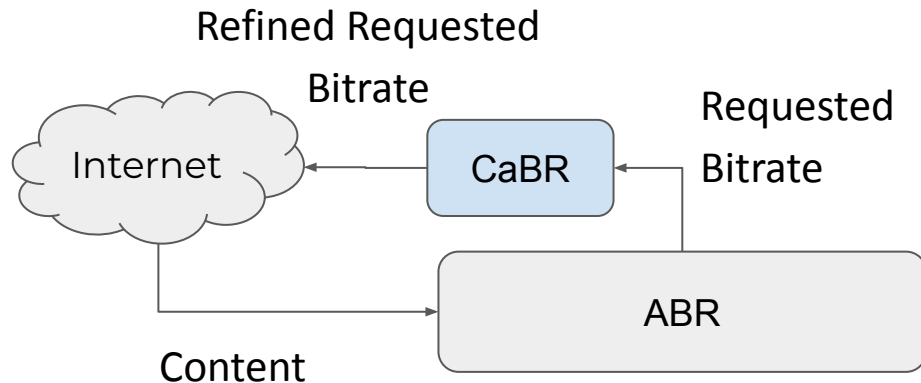
Congestion-aware adaptive BitRate algorithm (CaBR)

Problem

Traditional ABRs rely on delayed, application-level metrics (throughput, buffer).

CaBR

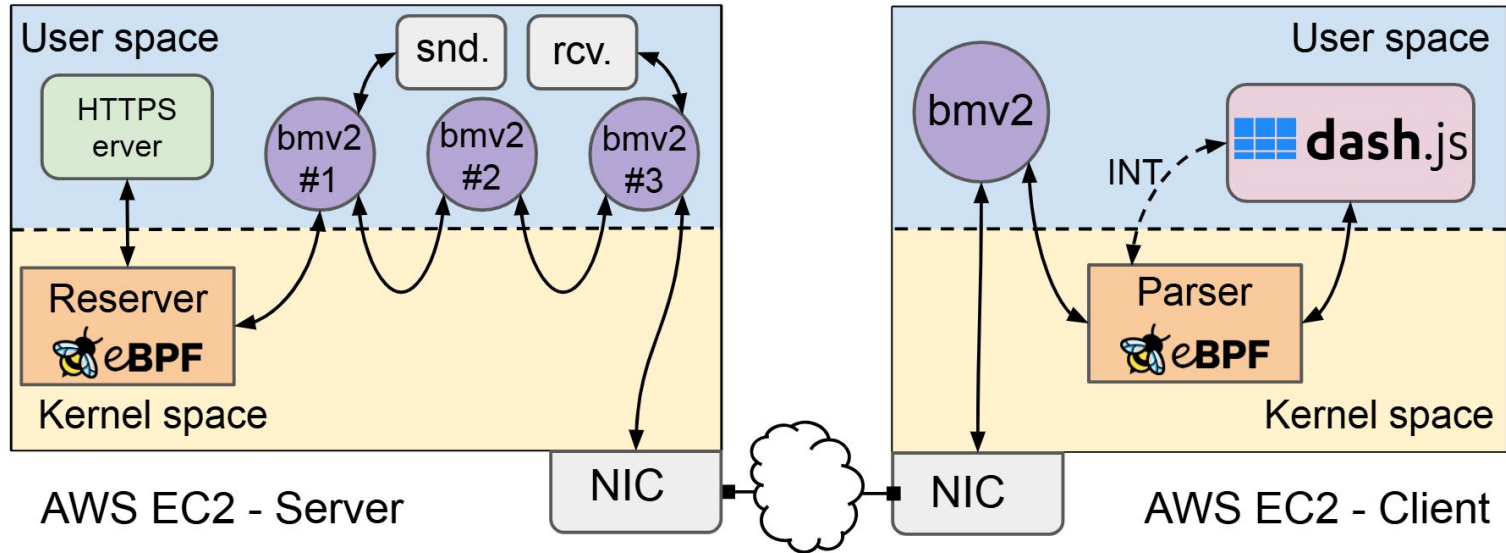
Relies on real-time, kernel-level congestion metrics (queue length) gathered along the network path using **In-band Network Telemetry (INT)**.



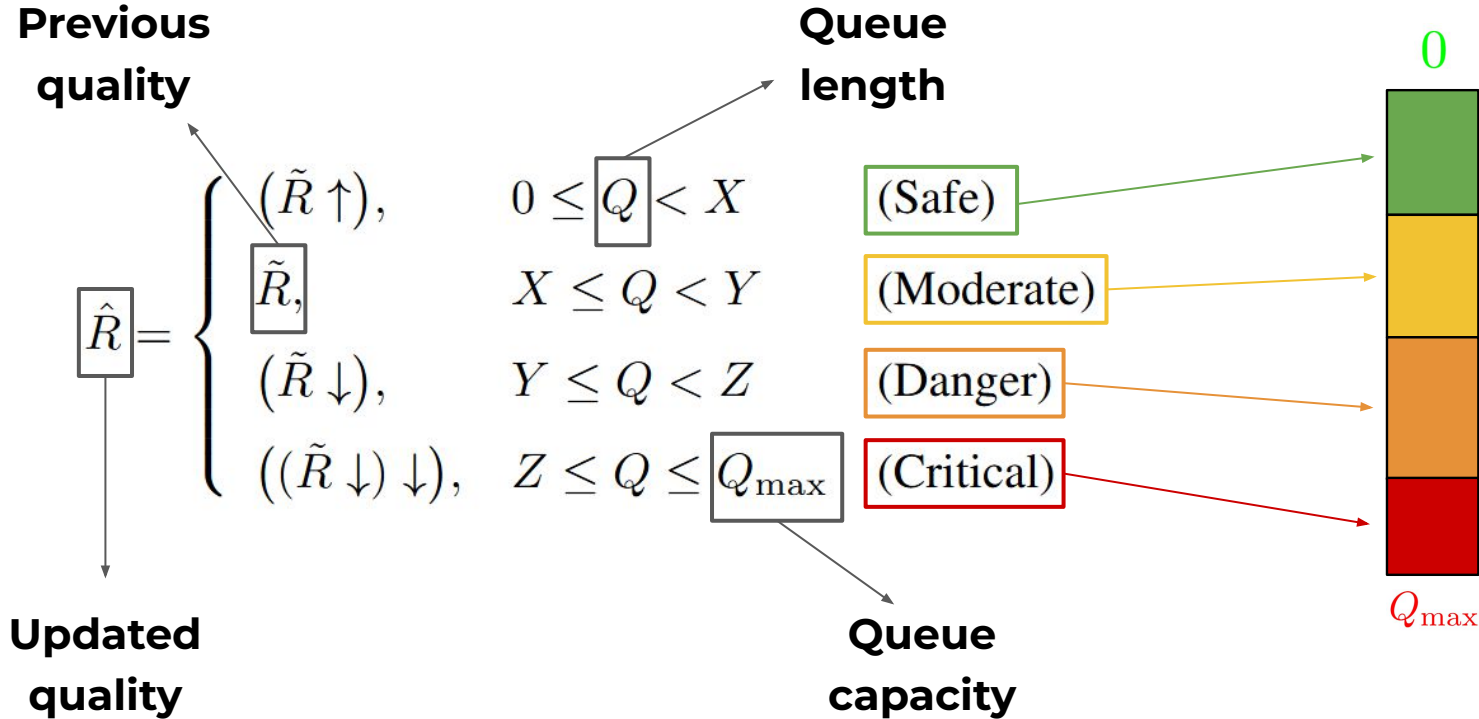
CaBR injects **cross-layer feedback**—specifically **queue length** from network switches—into the bitrate decision process.

eBPF-based telemetry

- Embedded in the Linux kernel.
- Captures per-packet **queue length** using **In-band Network Telemetry (INT)**.
- Exposes metrics to the client through **custom TCP options**.



CaBR Plugin Approach



Evaluation and Results



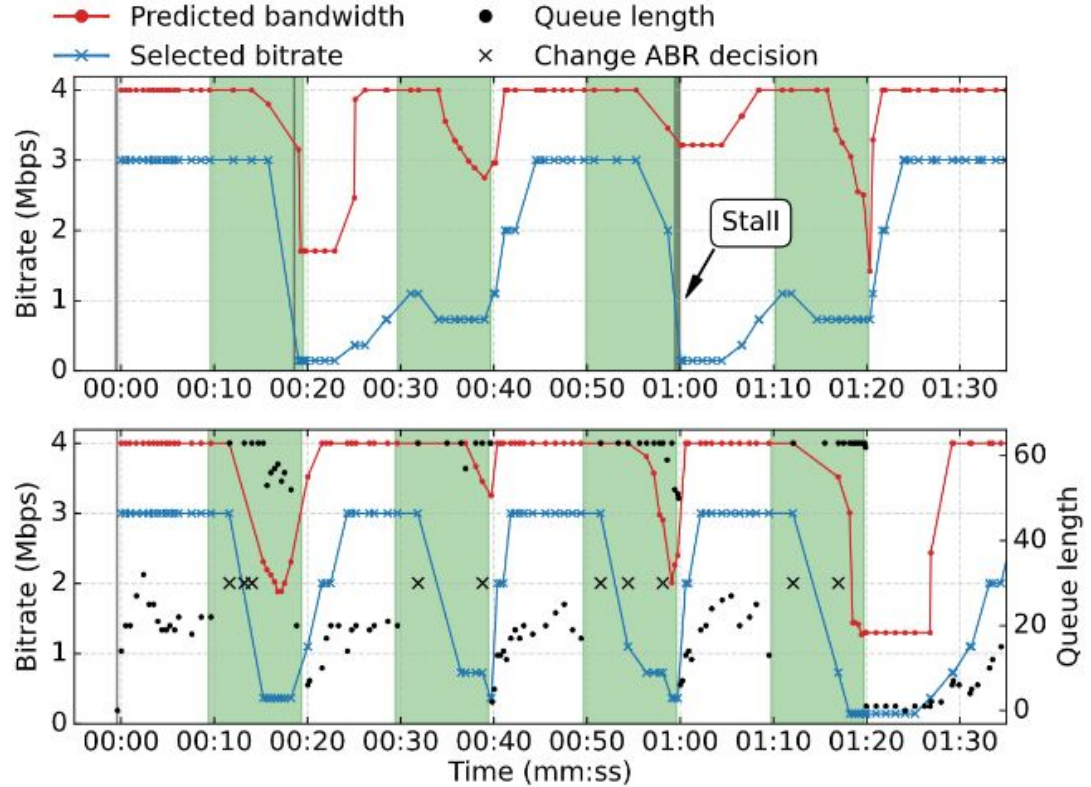
Evaluation setup

- **Bitrate ladder:**
 - {0.1, 0.2, 0.375, 0.55, 0.75, 1, 1.5, 3, 5.8} Mbps;
 - {256x144, 320x180, 384x216, 512x288, 640x360, 768x432, 1024x576, 1280x720, 1920x1080};
- **Codec:** Advanced Video Codec (AVC);
- **Video sequences:** Big Buck Bunny (first 270 s);
- **Segment length:** 1s;
- **Buffer capacity:** 20s;
- **Player:** dashjs;
- **QoE model:** ITU-T P.1203 (mode 0).
- **Scenario:** 100s test executed 5 times → 4 load bursts with 10s duration and 10s offset;



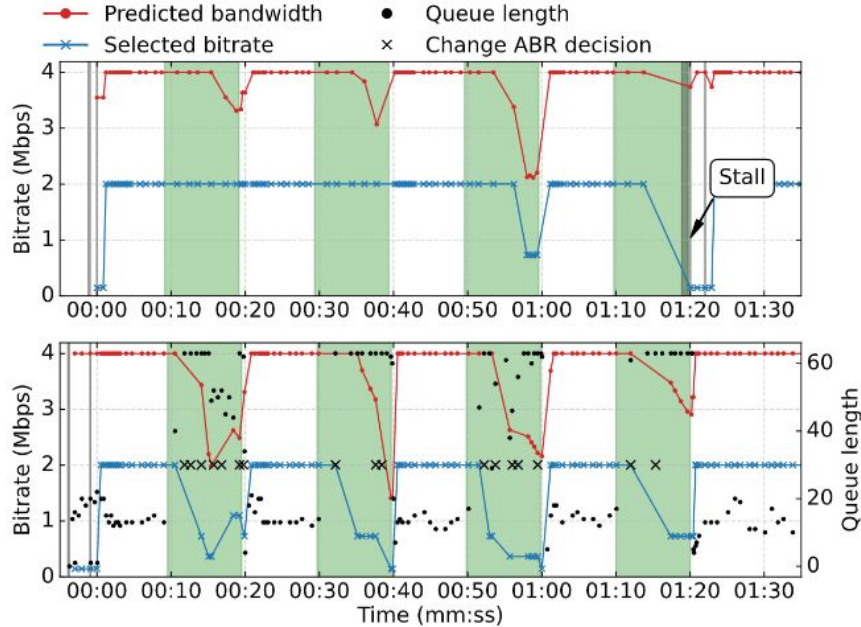
Results

- Top row: original approach;
bottom row: with CaBR;
- **Prevents all stalls** from happening;
- Adapts (even aggressively) the requested bitrate based on the queue length.

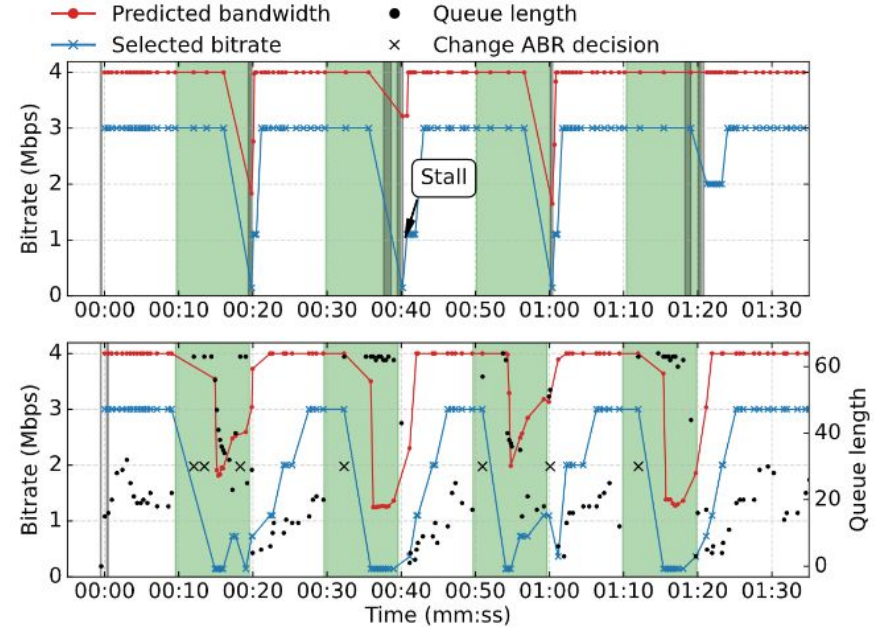


(a) BOLA

Results



(b) LoL+

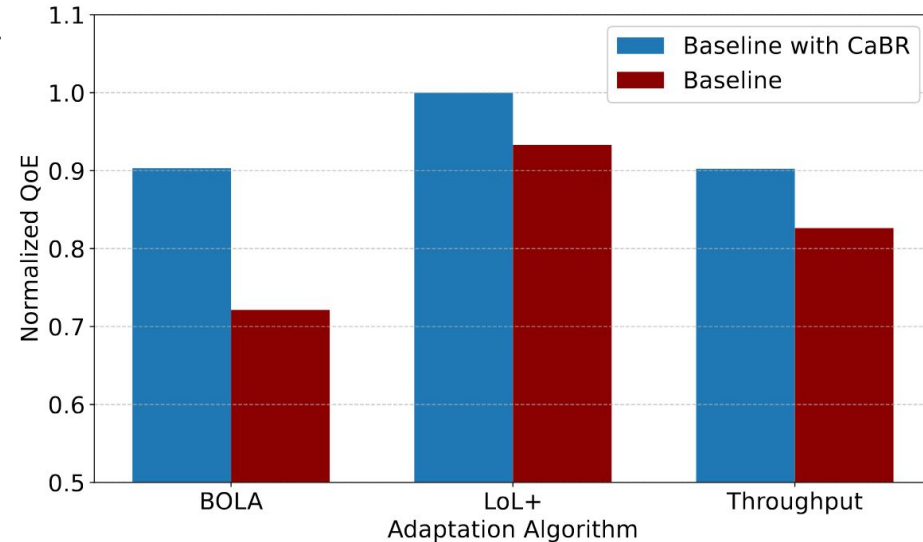


(c) Throughput-based ABR

Results

- Reducing stalls
- Increasing QoE;
- CaBR:
 - QoE increased by **25%** (BOLA);
 - QoE increased by **8%** (LoL+);
 - QoE increased by **10%** (Throughput).

Normalized QoE comparison of CaBR with state-of-the-art baselines (↑)



Conclusion

- **Key points:**
 - **CaBR** effectively **reacts to congestion** changes thanks to queue length by **eBPF**;
 - **Stalls avoided completely** in our experiments;
 - Can achieve up to **25% QoE** increase compared to state-of-the-art ABR algorithms.
- **Drawbacks:**
 - Need switches/routers with eBPF running;
 - Client must fetch eBPF parameters through interlayer applications (user/kernel space).
- **Future work:**
 - Exploit other eBPF parameters (i.e. latency, delays, etc.);
 - Compare the performance of eBPF with L4S architecture

Thank you for your attention!

Q&A

Receiving Kernel-Level Insights via eBPF: Can ABR Algorithms Adapt Smarter?

Thank you for your attention!



WueWoWAS 2025